

Documentación Técnica: Modelos Fundacionales para Series Temporales e Infraestructura GPU

Trabajo de Fin de Grado — Predicción de Fatiga con FATIGUESet

Emilio Gullón

Julio 2026

Resumen

Este documento recoge la implementación, despliegue y evaluación experimental de tres modelos fundacionales para series temporales — MOMENT (encoder-only), CHRONOS (decoder-only probabilístico) y TIMESFM (decoder-only con parches) — aplicados a la predicción de fatiga física y mental a partir de señales fisiológicas del dataset FATIGUESet. Se describe en detalle la librería `fatigueset.models.foundation`, el módulo de optimización de hiperparámetros con Optuna (`fatigueset.models.optimizers`), la infraestructura de ejecución en un clúster GPU universitario (UGR), y los resultados comparativos frente a modelos clásicos de Machine Learning y redes de Deep Learning diseñadas a medida. Adicionalmente, se documentan los problemas técnicos encontrados (errores de memoria GPU, límites de tiempo en SLURM) y las soluciones aplicadas.

Índice

1. Introducción	3
1.1. Contexto del TFG y FatigueSet	3
1.2. Motivación: modelos fundacionales para series temporales	3
1.3. Objetivos de esta fase experimental	3
2. Marco Teórico: Modelos Fundacionales para Series Temporales	4
2.1. Taxonomía de arquitecturas	4
2.2. MOMENT: arquitectura, parches y preentrenamiento	4
2.3. Chronos: tokenización por quantile binning y T5	5
2.4. TimesFM: parches de longitud variable y decoder patched	6
3. Modelos Clásicos de Aprendizaje Automático	6
3.1. Protocolo Experimental y Preprocesamiento	6
3.2. Ranking y Comparativa de Rendimiento	7
3.3. Análisis del Impacto de la Normalización por Sujeto (Subject-wise Normalization)	7
3.4. Importancia de Características (Feature Importance)	8
4. Modelos de Deep Learning Diseñados a Medida	9
4.1. Resumen de Arquitecturas Diseñadas	9
4.2. Extended Long Short-Term Memory (xLSTM / sLSTM)	10

4.2.1.	Formulación Matemática de la Celda sLSTM	10
4.2.2.	Implementación en la Biblioteca	11
5.	Implementación de la Librería <code>fatigueset.models.foundation</code>	12
5.1.	<code>MOMENTFatigueRegressor</code> : diseño y forward pass	12
5.2.	<code>ChronosZeroShotEvaluator</code> : sonda lineal sobre features zero-shot	12
5.3.	<code>TimesFMZeroShotEvaluator</code> : cuantiles y predicción por canal	13
5.4.	Métricas probabilísticas: CRPS y cobertura	13
6.	Módulo de Optimizadores (<code>fatigueset.models.optimizers</code>)	14
6.1.	Arquitectura del módulo	14
6.2.	Búsqueda condicional con Optuna	14
6.3.	Espacios de búsqueda por familia de modelo	15
7.	Infraestructura de Computación en Servidor GPU	16
7.1.	Arquitectura del clúster UGR	16
7.2.	Entorno Conda remoto	16
7.3.	Scripts SLURM de ejecución	16
7.4.	Pipeline de despliegue automático	18
7.5.	Monitorización remota de jobs	18
8.	Resultados Experimentales	18
8.1.	Tabla comparativa global	18
8.2.	<code>MOMENT</code> : resultados por fold	19
8.3.	<code>Chronos</code> : métricas zero-shot y comparación de escalas	20
8.4.	<code>TimesFM</code> : métricas puntuales y probabilísticas	21
8.5.	<code>Optuna</code> : resultados del experimento básico	21
8.6.	Discusión y análisis comparativo	22
9.	Problemas Técnicos y Soluciones	22
9.1.	CUDA Out of Memory: diagnóstico y corrección	22
9.2.	Exceso de Cuota de Disco en Directorio Home (Disk Quota Exceeded)	24
9.3.	Exceso de tiempo en SLURM (<code>Optuna</code>)	25
10.	Conclusiones y Trabajo Futuro	25
10.1.	Comparativa final de paradigmas	25
10.2.	Tareas pendientes	25

1. Introducción

1.1. Contexto del TFG y FatigueSet

Este Trabajo de Fin de Grado aborda la predicción cuantitativa de fatiga física y mental a partir de señales fisiológicas multicanal del dataset FATIGUESet [1]. El dataset contiene registros de 12 participantes monitorizados con dispositivos portátiles (Empatica E4 y Zephyr BioHarness) durante tareas cognitivas y físicas, proporcionando 23 canales fisiológicos (acelerómetros, electrodermal, temperatura, ritmo cardíaco, respiración, etc.) junto con puntuaciones subjetivas de fatiga en escala 0–100.

El pipeline experimental se estructura en cuatro fases progresivas de complejidad:

1. **Modelos clásicos de Machine Learning:** Random Forest, KNN, Lasso, Ridge, SVM, Decision Tree, ElasticNet y Regresión Lineal.
2. **Modelos de Deep Learning diseñados a medida:** LSTM, GRU, CNN-LSTM, TCN, Transformer, PatchTST y xLSTM.
3. **Optimización de hiperparámetros con Optuna:** Búsqueda bayesiana con TPE (Tree-structured Parzen Estimator) y optimizadores personalizados.
4. **Modelos fundacionales preentrenados:** MOMENT, CHRONOS y TIMESFM.

1.2. Motivación: modelos fundacionales para series temporales

Los modelos fundacionales (*Foundation Models*) representan un cambio de paradigma en el análisis de series temporales. Frente al enfoque tradicional de diseñar y entrenar un modelo específico para cada tarea, estos modelos se preentrenan sobre grandes corpora de series temporales diversas y posteriormente se adaptan a tareas downstream con mínima supervisión.

El survey de Kottapalli et al. [2] identifica tres paradigmas de adaptación:

- **Zero-shot:** el modelo preentrenado se aplica directamente sin ningún reentrenamiento.
- **Fine-tuning:** se reentrenan las últimas capas o una cabeza de tarea ligera.
- **Prompting / In-context learning:** se proporcionan ejemplos de demostración al modelo.

En este TFG se aplican los tres modelos fundacionales más relevantes identificados en el survey, evaluando su capacidad como extractores de características universales para la predicción de fatiga en un dominio no visto durante su preentrenamiento.

1.3. Objetivos de esta fase experimental

Los objetivos específicos de la fase de modelos fundacionales son:

- (a) Implementar adaptadores para MOMENT, CHRONOS y TIMESFM dentro de la librería modular `fatigueset`.
- (b) Evaluar el rendimiento de cada modelo en predicción de fatiga física y mental usando validación cruzada GroupKFold por participante.

- (c) Comparar métricas deterministas (MAE, RMSE, R^2) y probabilísticas (CRPS, cobertura del intervalo de predicción al 90 %).
- (d) Desplegar y ejecutar los experimentos a escala completa en un clúster GPU universitario.

2. Marco Teórico: Modelos Fundacionales para Series Temporales

2.1. Taxonomía de arquitecturas

Siguiendo la taxonomía del survey de Kottapalli et al. [2], los modelos fundacionales para series temporales se clasifican según su arquitectura base:

- **Encoder-only**: procesan la secuencia completa bidireccionalmente para producir representaciones contextualizadas. Ejemplo: MOMENT.
- **Decoder-only**: procesan la secuencia de forma autorregresiva (causal), prediciendo el siguiente token/valor. Ejemplos: CHRONOS, TIMESFM.
- **Encoder-Decoder**: combinan ambos enfoques. Ejemplo: Lag-Llama (no implementado en este TFG).

La Tabla 1 resume la taxonomía de los modelos implementados.

Cuadro 1: Taxonomía de los modelos fundacionales implementados en el TFG.

Modelo	Tipo	Patch-based	Multivariado	Probabilístico	Paradigma
MOMENT	Encoder-only	Sí	Sí	No	Fine-tuning cabeza
CHRONOS	Decoder-only	No	No	Sí	Zero-shot + probe
TIMESFM	Decoder-only	Sí	No	Sí	Zero-shot + probe

2.2. MOMENT: arquitectura, parches y preentrenamiento

MOMENT (*A Family of Open Time-series Foundation Models*) [4] es un modelo encoder-only basado en Transformer preentrenado mediante reconstrucción de parches enmascarados (*Masked Patch Reconstruction*), análogo al preentrenamiento de BERT en NLP pero adaptado a series temporales.

Arquitectura. La secuencia temporal de entrada $\mathbf{x} \in \mathbb{R}^{T \times C}$ (con T pasos temporales y C canales) se segmenta en *parches* no superpuestos de longitud fija P . Cada parche se proyecta a un espacio de dimensión d_{model} mediante una capa lineal, produciendo una secuencia de tokens:

$$\mathbf{z}_i = W_{\text{patch}} \cdot \mathbf{x}_{[iP:(i+1)P]} + \mathbf{b}_{\text{patch}}, \quad i = 0, 1, \dots, \lfloor T/P \rfloor - 1$$

Los tokens se procesan por L capas de Transformer encoder (auto-atención multi-cabeza + MLP feed-forward), produciendo representaciones contextualizadas $\mathbf{h}_i$ para cada parche.

Preentrenamiento. MOMENT se preentrena en el corpus *Time Series Pile*, una colección de más de 1 billón de observaciones de series temporales de dominios diversos (finanzas, salud, clima, energía). El objetivo de preentrenamiento es la reconstrucción de parches enmascarados: se enmascara un porcentaje aleatorio de parches y el modelo debe reconstruirlos a partir del contexto.

Variantes. Se implementan dos variantes:

- **MOMENT-1-small:** 40M parámetros (pruebas locales).
- **MOMENT-1-large:** 385M parámetros (experimento en servidor, 341.250.570 parámetros totales).

Adaptación a FatigueSet. Para adaptar MOMENT a la predicción de fatiga, se congela el backbone preentrenado y se añade una cabeza de regresión lineal $\text{Linear}(d_{\text{model}}, 2)$ que mapea la representación agregada (media de los embeddings de todos los parches) a las dos dimensiones de fatiga. Solo se entrenan los 2.050 parámetros de la cabeza (0.00 % del total), lo que reduce drásticamente el riesgo de sobreajuste en el pequeño dataset de FATIGUSET.

2.3. Chronos: tokenización por quantile binning y T5

CHRONOS [5] es un framework probabilístico decoder-only para series temporales que reutiliza la arquitectura T5 (*Text-to-Text Transfer Transformer*) de Google, adaptándola al dominio temporal mediante una estrategia de tokenización numérica.

Tokenización. Los valores continuos de la serie temporal se discretizan en *bins* mediante *quantile binning*: se calculan los cuantiles empíricos del contexto y cada valor se asigna al bin correspondiente. Esta estrategia permite reutilizar el vocabulario discreto del T5 sin modificar la arquitectura.

Preentrenamiento. CHRONOS se preentrena con un objetivo de lenguaje estándar (predicción del siguiente token) sobre el corpus TSMixup, una mezcla de series temporales reales y sintéticas generadas por Gaussian Processes.

Variantes.

- **chronos-t5-tiny:** 8M parámetros (pruebas locales).
- **chronos-t5-large:** 710M parámetros (experimento en servidor).

Adaptación a FatigueSet. Dado que CHRONOS está preentrenado exclusivamente para *forecasting* univariado, la adaptación a regresión multidimensional de fatiga se realiza mediante un enfoque de **linear probe** (sonda lineal):

1. Cada uno de los 23 canales fisiológicos se trata como una serie temporal univariada independiente.
2. CHRONOS predice la distribución del siguiente paso para cada canal, obteniéndose la mediana como característica.

3. Las 23 medianas se concatenan como vector de características y se alimentan a una regresión Ridge que mapea al espacio de fatiga bidimensional.

2.4. TimesFM: parches de longitud variable y decoder patched

TIMESFM 2.5 (*Time Series Foundation Model*) [6] es un modelo decoder-only de Google Research basado en parches de longitud variable. A diferencia de MOMENT, que usa parches de longitud fija, TIMESFM utiliza un mecanismo de *input patching* con múltiples resoluciones que permite procesar series temporales de diferentes frecuencias sin re-muestreo.

Arquitectura. TIMESFM 2.5 utiliza un decoder Transformer con 200M de parámetros, entrenado sobre 100 billones de observaciones de series temporales reales y sintéticas. Su diseño clave es la predicción por parches con tamaño de salida variable (*output patch size*), lo que lo hace extremadamente eficiente en inferencia.

Adaptación a FatigueSet. La adaptación sigue el mismo paradigma de channel independence y linear probe que CHRONOS:

1. Los 23 canales se procesan independientemente por TIMESFM.
2. El modelo genera predicciones puntuales y distribuciones de cuantiles (10 cuantiles predefinidos) por canal.
3. Las predicciones puntuales se concatenan como vector de 23 características y se alimentan a una regresión Ridge.
4. Los cuantiles se usan para calcular métricas probabilísticas (CRPS, cobertura).

3. Modelos Clásicos de Aprendizaje Automático

Esta sección documenta los experimentos realizados con algoritmos de aprendizaje automático clásico (Machine Learning clásico). Estos modelos se entrenaron sobre un dataset estructurado tabular obtenido mediante agregación temporal de las señales fisiológicas.

3.1. Protocolo Experimental y Preprocesamiento

Para los modelos clásicos, se estructuró el dataset tabular `df_ml` que consta de 108 filas (correspondientes a los 12 participantes, evaluados en 3 sesiones independientes con 3 fases experimentales cada una). Cada muestra contiene más de 50 características agregadas que resumen los estadísticos (media, desviación estándar) de los sensores cardíacos (frecuencia cardíaca, variabilidad cardíaca, respiración), sensores periféricos de la muñeca (temperatura, conductancia de la piel) y bandas de potencia de EEG.

La evaluación se realizó utilizando un protocolo de validación cruzada de 5 particiones (KFold aleatorio, $k = 5$, con semilla fija para reproducibilidad). Se entrenaron los siguientes regresores de la librería `scikit-learn`:

- **Random Forest Regressor:** Ensamble por *bagging* con 50 árboles de decisión y profundidad máxima de 10.

- **K-Nearest Neighbors (KNN)**: Regresor basado en vecindad con pesos uniformes ($k = 3$ y $k = 5$).
- **Lasso Regressor**: Regresión lineal con regularización L_1 ($\alpha = 0,1$).
- **Decision Tree Regressor**: Árbol de decisión simple sin límite de profundidad (utilizado como baseline de sobreajuste).
- **ElasticNet**: Regresión lineal regularizada con penalización combinada L_1 y L_2 ($\alpha = 0,1$).
- **Ridge Regressor**: Regresión lineal regularizada con penalización L_2 ($\alpha = 1,0$).
- **Support Vector Regressor (SVR)**: Máquina de vectores de soporte con kernel de función de base radial (RBF) y constante de penalización $C = 100$.
- **Regresión Lineal**: Regresión por mínimos cuadrados ordinarios sin regularización.

3.2. Ranking y Comparativa de Rendimiento

Los modelos se evaluaron usando R^2 de validación cruzada y el MAE medio sobre el conjunto completo de entrenamiento. En la Tabla 2 se muestra el ranking de los modelos para la fatiga física.

Cuadro 2: Ranking y comparación de modelos clásicos para Fatiga Física (CV de 5 splits).

Modelo	Tipo	R^2 CV Medio	Desv. Est. R^2 CV	MSE Train	MAE Train
Random Forest	Ensamble	-2.4084	2.6050	1.0772	0.8767
KNN (k=3)	Instancias	-3.5707	3.2852	4.8593	1.6889
Lasso ($\alpha = 0,1$)	Regularizado	-3.8720	3.7321	8.2614	2.3716
Decision Tree	Árbol	-4.7405	6.6825	0.0000	0.0000
KNN (k=5)	Instancias	-4.8633	3.4261	8.5387	2.3467
ElasticNet	Regularizado	-5.7379	7.3465	11.3750	2.9468
Ridge ($\alpha = 1,0$)	Regularizado	-6.0179	7.9491	11.3456	2.9367
SVR	Kernel	-6.5572	8.6872	7.6892	2.1611
Regresión Lineal	Lineal	-12.5545	15.5509	5.5752	1.9046

3.3. Análisis del Impacto de la Normalización por Sujeto (Subject-wise Normalization)

Una de las propuestas de preprocesamiento analizadas para mitigar la heterogeneidad individual (diferencias de línea base y rango dinámico de las señales fisiológicas entre participantes) fue la aplicación de una **Normalización por Participante y Sesión (Subject-Session-wise Normalization)**.

Formulación Matemática. Para cada participante i y sesión s , cada característica x se escala de forma independiente mediante Z-score utilizando únicamente la media $\mu_{i,s}$ y la desviación estándar $\sigma_{i,s}$ de ese bloque de datos:

$$z_{i,s,t} = \frac{x_{i,s,t} - \mu_{i,s}}{\sigma_{i,s}}$$

Donde t representa los pasos temporales dentro de la sesión.

Evaluación Empírica. Para cuantificar rigurosamente su impacto, se realizó un experimento comparativo utilizando un modelo robusto de Random Forest bajo validación cruzada GroupKFold por participante (garantizando que ningún dato de los participantes de test se use durante el entrenamiento). Los resultados obtenidos para la predicción de fatiga física son:

- **Sin Normalización:** $MAE = 2,6878 \mid R^2 = 0,4120$
- **Con Normalización por Sujeto/Sesión:** $MAE = 3,9222 \mid R^2 = -0,2614$

El uso de la normalización por participante y sesión provocó un incremento del **45.9 % en el error absoluto medio (MAE)** y redujo el coeficiente de determinación R^2 de una puntuación muy positiva (0.41) a una negativa (-0.26), destruyendo la capacidad predictiva del modelo.

Justificación del Empeoramiento. El fracaso de esta normalización se debe a tres factores clave:

1. **Pérdida de la varianza inter-sesión (línea base de fatiga):** Al centrar a cero cada sesión por separado, se elimina la diferencia de escala absoluta entre una sesión en la que el sujeto está relajado (frecuencia cardíaca baja) y una sesión de esfuerzo o fatiga (frecuencia cardíaca alta). Si ambas sesiones se escalan para tener media cero, sus representaciones se vuelven casi idénticas a ojos del modelo, imposibilitando predecir la diferencia de fatiga.
2. **Amplificación del ruido:** En sesiones de muy baja actividad o estados muy estables, la desviación estándar $\sigma_{i,s}$ es extremadamente pequeña. Al dividir por un valor cercano a cero, cualquier ruido térmico o pequeña fluctuación de lectura del sensor se magnifica en el Z-score, actuando como ruido severo.
3. **Pérdida de leyes biológicas universales:** Las respuestas físicas humanas ante la fatiga siguen tendencias absolutas (p. ej., a mayor fatiga, mayor ritmo cardíaco y mayor conductancia galvánica de la piel). La normalización local descarta esta escala absoluta común, dificultando que el modelo generalice sus conocimientos a nuevos sujetos invisibles.

3.4. Importancia de Características (Feature Importance)

El modelo **Random Forest Regressor** permite extraer la importancia relativa de las variables basándose en la reducción de impureza (Gini/varianza). La Tabla 3 presenta las 15 características fisiológicas más importantes identificadas por el modelo para predecir la fatiga física.

Cuadro 3: Top 15 características fisiológicas más importantes (Random Forest).

Posición	Característica	Importancia Relativa
1	eda_media (wrist)	0.1319
2	hr_media (chest)	0.1217
3	eeg_beta_media (EEG)	0.0971
4	br_media (chest)	0.0839
5	eda_std (wrist)	0.0750
6	hr_std (chest)	0.0679
7	hrv_media (chest)	0.0667
8	delta_fisica_cognitivo (diferencia)	0.0566
9	hrv_std (chest)	0.0499
10	br_std (chest)	0.0442
11	eeg_alpha_media (EEG)	0.0438
12	eeg_theta_media (EEG)	0.0413
13	delta_mental_cognitivo (diferencia)	0.0300
14	delta_mental_ejercicio (diferencia)	0.0237
15	mental_M3 (estadístico)	0.0233

Análisis de características. La alta relevancia de `eda_media` y `hr_media` concuerda con la teoría fisiológica de la fatiga, donde la conductancia galvánica y la frecuencia cardíaca actúan como marcadores directos del estrés y la carga cardiovascular. Asimismo, la presencia de las bandas beta y alfa de EEG evidencia la implicación del cansancio neurológico en las fases cognitivas del experimento.

4. Modelos de Deep Learning Diseñados a Medida

En esta fase del proyecto, para modelar la dinámica temporal compleja de las señales fisiológicas multicanal, se diseñaron e implementaron en PyTorch siete arquitecturas de Deep Learning a medida. Estas redes se entrenaron utilizando secuencias multivariadas de timesteps correspondientes a ventanas temporales del dataset, mapeando la entrada de forma bidimensional hacia las etiquetas continuas de fatiga física y mental.

4.1. Resumen de Arquitecturas Diseñadas

Las arquitecturas propuestas abarcan un amplio espectro de paradigmas de procesamiento temporal:

- **Custom LSTM y GRU:** Modelos recurrentes estándar para capturar dependencias secuenciales a largo plazo.
- **Custom CNN-LSTM:** Modelo híbrido que combina capas convolucionales 1D para la extracción de características espaciales locales y capas recurrentes LSTM para capturar la evolución temporal.
- **Custom TCN (Temporal Convolutional Networks):** Arquitectura convolucional que emplea convoluciones causales y dilatadas junto con conexiones residuales, ofreciendo un campo receptivo amplio y evitando el desvanecimiento del gradiente sin la necesidad de recurrencia.

- **Custom Transformer**: Modelo basado en auto-atención multi-cabeza y codificación posicional para capturar relaciones globales sin restricciones secuenciales.
- **Custom PatchTST**: Adaptación de la arquitectura de Transformer que opera sobre parches o segmentos locales en lugar de timesteps individuales, e implementa independencia de canales (*channel independence*).
- **Custom xLSTM (sLSTM)**: Red recurrente avanzada basada en el reciente paradigma xLSTM, implementada de forma manual para sortear las limitaciones de las compuertas sigmoideas tradicionales.

4.2. Extended Long Short-Term Memory (xLSTM / sLSTM)

El modelo **xLSTM** (Extended Long Short-Term Memory) [3] introduce mejoras fundamentales sobre la LSTM clásica mediante el uso de compuertas exponenciales y mecanismos de estabilización numérica. En este trabajo, ante la ausencia de una implementación oficial compatible en la librería estándar de PyTorch, se desarrolló manualmente una variante **sLSTM** (Stabilized LSTM) en el módulo `xlstm.py`.

4.2.1. Formulación Matemática de la Celda sLSTM

A diferencia de la LSTM clásica, que utiliza funciones sigmoideas para las compuertas de entrada y olvido (limitando su rango a $(0, 1)$), la variante sLSTM emplea funciones exponenciales para permitir que la red actualice sus estados con mayor flexibilidad. Sin embargo, el uso directo de exponenciales conlleva riesgos de desbordamiento numérico (*overflow*). Para resolver esto, sLSTM introduce un estado estabilizador m_t y un normalizador n_t .

Las ecuaciones que describen el paso hacia adelante (*forward pass*) para un paso temporal t en la celda sLSTM son:

1. Pre-activaciones lineales:

$$\tilde{f}_t = x_t W_f^T + h_{t-1} U_f^T + b_f \quad (1)$$

$$\tilde{i}_t = x_t W_i^T + h_{t-1} U_i^T + b_i \quad (2)$$

$$\tilde{c}_t = \tanh(x_t W_c^T + h_{t-1} U_c^T + b_c) \quad (3)$$

$$\tilde{o}_t = x_t W_o^T + h_{t-1} U_o^T + b_o \quad (4)$$

Donde $\tilde{f}_t$, $\tilde{i}_t$, $\tilde{c}_t$ y $\tilde{o}_t$ son las pre-activaciones de la compuerta de olvido, compuerta de entrada, candidato de celda y compuerta de salida, respectivamente. Los pesos W_* y U_* y sesgos b_* son parámetros aprendidos.

2. **Estabilización en escala logarítmica**: Para evitar el desbordamiento numérico al calcular los exponenciales de las compuertas, se calcula un término de escala m_t que mantiene el exponente máximo observado:

$$m_t = \max(m_{t-1} + \tilde{f}_t, \tilde{i}_t) \quad (5)$$

Con una condición de inicialización $m_0 = 0$.

3. **Compuertas Exponenciales Estabilizadas:** Las compuertas de olvido y de entrada se calculan en base a la escala de estabilización m_t :

$$f_t^s = \exp(m_{t-1} + \tilde{f}_t - m_t) \quad (6)$$

$$i_t^s = \exp(\tilde{i}_t - m_t) \quad (7)$$

De esta forma, la suma de los exponentes está acotada superiormente por 0, evitando de forma robusta desbordamientos numéricos por arriba.

4. **Actualización del Estado de Celda y Normalizador:** El estado interno de la celda c_t y el normalizador acumulativo n_t se actualizan secuencialmente:

$$c_t = f_t^s \odot c_{t-1} + i_t^s \odot \tilde{c}_t \quad (8)$$

$$n_t = f_t^s \odot n_{t-1} + i_t^s \quad (9)$$

Donde $\odot$ denota el producto de Hadamard (elemento a elemento). El normalizador se inicializa como $n_0 = 1$.

5. **Normalización del Estado de Celda y Salida:** Para contrarrestar el crecimiento debido a los términos exponenciales, el estado de la celda se normaliza antes de proyectarlo al estado oculto:

$$\hat{c}_t = \frac{c_t}{n_t + \epsilon} \quad (10)$$

Donde $\epsilon = 10^{-8}$ es una constante para evitar divisiones por cero. Finalmente, el estado oculto h_t se obtiene como:

$$o_t = \sigma(\tilde{o}_t) \quad (11)$$

$$h_t = o_t \odot \tanh(\hat{c}_t) \quad (12)$$

4.2.2. Implementación en la Biblioteca

La celda anterior se programó en la clase `CustomxLSTMCell` dentro de `fatigueset/models/xlstm.py`. Posteriormente, la clase `CustomxLSTM` define la pila de capas recurrentes, permitiendo conectar múltiples capas de celdas sLSTM y aplicando regularización por *dropout* entre capas intermedias. La clase final `CustomxLSTMRegressor` hereda de `nn.Module` y mapea el último paso temporal de la secuencia mediante una proyección lineal al espacio continuo de salida bidimensional de la fatiga:

```

1 class CustomxLSTMRegressor(nn.Module):
2     def __init__(self, input_size: int, hidden_size: int = 64,
3         num_layers: int = 2, dropout: float = 0.2, output_size:
4         int = 2):
5         super().__init__()
6         self.xlstm = CustomxLSTM(input_size, hidden_size,
7             num_layers, dropout)
8         self.fc = nn.Linear(hidden_size, output_size)
9
10    def forward(self, x: torch.Tensor) -> torch.Tensor:
11        # x: (batch_size, seq_len, input_size)
12        out, _ = self.xlstm(x)
13        last_step_out = out[:, -1, :]
14        return self.fc(last_step_out)

```

Listing 1: Estructura de la clase CustomxLSTMRegressor.

5. Implementación de la Librería `fatigueset.models.foundation`

El módulo `foundation.py` (924 líneas, 34.6 KB) implementa toda la lógica de adaptación de los tres modelos fundacionales. Se estructura en cinco componentes principales.

5.1. MOMENTFatigueRegressor: diseño y forward pass

La clase `MOMENTFatigueRegressor` hereda de `torch.nn.Module` y encapsula el backbone `MOMENT` con una cabeza de regresión personalizada.

```
1 class MOMENTFatigueRegressor(nn.Module):
2     def forward(self, x: torch.Tensor) -> torch.Tensor:
3         # Transponer: (B, seq_len, n_channels) -> (B, n_channels
4         #           , seq_len)
5         x_moment = x.transpose(1, 2).contiguous()
6
7         # Embeddings preentrenados (backbone congelado)
8         outputs = self._backbone.embed(x_enc=x_moment, reduction
9         #           ="mean")
10        pooled = outputs.embeddings # (B, d_model)
11
12        # Cabeza de regresion entrenada
13        return self.regression_head(self.dropout_layer(pooled))
```

Listing 2: Paso forward de `MOMENTFatigueRegressor` (simplificado).

Diseño de la cabeza. La cabeza de regresión consiste en:

- `Dropout(p=0.1)`: regularización para evitar sobreajuste.
- `Linear( $d_{\text{model}}$ , 2)`: proyección lineal al espacio de fatiga bidimensional, con inicialización Xavier uniforme para los pesos y ceros para los sesgos.

Transposición de ejes. `MOMENT` espera la entrada en formato `(batch, n_channels, seq_len)`, mientras que el resto de modelos de `FATIGUSET` usan `(batch, seq_len, n_channels)`. El método `forward` realiza la transposición internamente, garantizando la compatibilidad sin modificar el pipeline de datos.

Carga perezosa del backbone. El backbone se carga de forma perezosa (*lazy loading*): si se proporciona un backbone pre-cargado en el constructor (parámetro `backbone`), se reutiliza directamente; en caso contrario, se carga automáticamente en el primer `forward` pass mediante `MOMENTPipeline.from_pretrained`.

5.2. ChronosZeroShotEvaluator: sonda lineal sobre features zero-shot

La clase `ChronosZeroShotEvaluator` implementa la evaluación zero-shot de `CHRONOS` con tres métodos principales:

1. `extract_features(X)`: itera sobre los 23 canales, llama a `predict_quantiles` de CHRONOS para cada canal, y extrae la mediana (cuantil 0.5) del siguiente paso como característica. Resultado: matriz $(N, 23)$.
2. `fit_probe(X_train, y_train, X_val)`: entrena una regresión Ridge ($\alpha = 1,0$) sobre las 23 características extraídas y predice sobre el conjunto de validación.
3. `predict_quantiles_for_crps(X)`: predice 9 cuantiles (0.1 a 0.9) por canal para el cálculo de métricas probabilísticas.

5.3. TimesFMZeroShotEvaluator: cuantiles y predicción por canal

La clase `TimesFMZeroShotEvaluator` sigue el mismo paradigma pero aprovecha la API nativa de TIMESFM 2.5:

```

1 def extract_features(self, X, batch_size=32):
2     N, seq_len, n_channels = X.shape
3     point_preds = np.zeros((N, n_channels))
4     quantile_preds = np.zeros((N, n_channels, 10))
5
6     for start in range(0, N, batch_size):
7         batch = X[start:end]
8         flat_inputs = [batch[i, :, ch]
9                        for i in range(B) for ch in range(
10                            n_channels)]
11
12         point_pred, quant_pred = self._model.forecast(
13             horizon=1, inputs=flat_inputs)
14         # Des-aplanar resultados...
15     return point_preds, quantile_preds

```

Listing 3: Extracción de características con TimesFM (simplificado).

La diferencia clave con CHRONOS es que TIMESFM genera directamente 10 cuantiles predefinidos en su salida nativa, sin necesidad de muestreo de Monte Carlo.

5.4. Métricas probabilísticas: CRPS y cobertura

Se implementan dos métricas probabilísticas estándar:

CRPS (Continuous Ranked Probability Score). Bajo el supuesto de distribución Normal, el CRPS se calcula analíticamente como [7]:

$$\text{CRPS}(\mu, \sigma, y) = \sigma \left[z(2\Phi(z) - 1) + 2\phi(z) - \frac{1}{\sqrt{\pi}} \right], \quad z = \frac{y - \mu}{\sigma}$$

donde $\Phi(\cdot)$ y $\phi(\cdot)$ son la función de distribución acumulada y la densidad de la Normal estándar, respectivamente.

El CRPS generaliza el MAE al espacio probabilístico: para predicciones deterministas ($\sigma \rightarrow 0$), el CRPS degenera al MAE estándar.

Cobertura del intervalo de predicción. La cobertura empírica mide la fracción de observaciones reales que caen dentro del intervalo de predicción:

$$\text{Coverage}_{1-\alpha} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[q_{\alpha/2}^{(i)} \leq y_i \leq q_{1-\alpha/2}^{(i)} \right]$$

Un intervalo de predicción del 90 % bien calibrado debería tener una cobertura empírica cercana a 0.90.

6. Módulo de Optimizadores (`fatigueset.models.optimizers`)

El módulo `optimizers.py` (477 líneas, 17.4 KB) extiende la búsqueda de hiperparámetros con Optuna al incluir la selección del **optimizador como hiperparámetro**.

6.1. Arquitectura del módulo

El módulo se estructura en cuatro componentes:

1. **build_optimizer:** Función de fábrica que instancia un optimizador de PyTorch a partir de su nombre y parámetros. Soporta Adam, AdamW, RMSprop y SGD (con momentum de Nesterov).
2. **sample_optimizer_params:** Define el espacio de búsqueda condicional del optimizador en Optuna. Primero se selecciona el tipo de optimizador como variable categórica, y luego se activan únicamente los hiperparámetros relevantes:
 - **Todos:** learning rate (10^{-5} a 10^{-2} , escala log) y weight decay (10^{-7} a 10^{-3} , escala log).
 - **SGD solamente:** momentum (0.7 a 0.99).
 - **RMSprop solamente:** alpha (0.9 a 0.999).
3. **Funciones sample_* por familia:** Una función de muestreo por cada familia de modelos (LSTM, GRU, CNN-LSTM, TCN, Transformer, PatchTST, xLSTM, Random Forest) que define el espacio de búsqueda completo de la arquitectura *incluyendo* el optimizador.
4. **sample_model_hyperparams:** Función unificada que permite la búsqueda jerárquica y condicional entre familias de modelos en un único estudio de Optuna.

6.2. Búsqueda condicional con Optuna

La búsqueda condicional es especialmente importante para evitar sugerir hiperparámetros irrelevantes (p. ej., momentum para Adam), lo que reduce la dimensionalidad efectiva del espacio de búsqueda y mejora la eficiencia del sampler TPE.

```
1 def sample_optimizer_params(trial, prefix):
2     optimizer_name = trial.suggest_categorical(
3         f"{prefix}_optimizer", ["Adam", "AdamW", "RMSprop", "SGD"])
```

```

4     lr = trial.suggest_float(f"{prefix}_lr", 1e-5, 1e-2, log=
      True)
5     weight_decay = trial.suggest_float(
6         f"{prefix}_weight_decay", 1e-7, 1e-3, log=True)
7
8     params = {"optimizer": optimizer_name, "lr": lr,
9              "weight_decay": weight_decay}
10
11     if optimizer_name == "SGD":
12         params["momentum"] = trial.suggest_float(
13             f"{prefix}_momentum", 0.7, 0.99)
14     elif optimizer_name == "RMSprop":
15         params["alpha"] = trial.suggest_float(
16             f"{prefix}_alpha", 0.9, 0.999)
17     return params

```

Listing 4: Búsqueda condicional de optimizador en Optuna.

6.3. Espacios de búsqueda por familia de modelo

La Tabla 4 resume los hiperparámetros buscados por Optuna para cada familia de modelo.

Cuadro 4: Resumen de hiperparámetros explorados por Optuna por familia de modelo.

Familia	Hiperparámetros explorados
Random Forest	n_estimators (50–300), max_depth, min_samples_split, min_samples_leaf, max_features
LSTM	hidden_size (16–256), num_layers (1–4), dropout, batch_size, optimizador
GRU	hidden_size (16–256), num_layers (1–4), dropout, batch_size, optimizador
CNN-LSTM	conv_channels, kernel_size, pool_size, hidden_size, num_layers, dropout, optimiza- dor
TCN	num_channels (5 configuraciones), kernel_size (2– 8), dropout, optimizador
Transformer	d_model (algebraicamente divisible), nhead, num_layers, dim_feedforward, dropout, optimi- zador
PatchTST	patch_len, stride, d_model, n_heads, num_layers, dim_ff, dropout, optimizador
xLSTM	hidden_size (32–256), num_layers (1–4), dropout, optimizador

Restricción de divisibilidad en Transformer y PatchTST. Para las arquitecturas basadas en auto-atención multi-cabeza, el parámetro d_{model} debe ser divisible por el número de cabezas (nhead). En lugar de imponer esta restricción mediante pruning

(descartando trials inválidos), se construye $d_{\text{model}} = \text{nhead} \times k$, donde k es un multiplicador entero sugerido por Optuna. Esto garantiza la divisibilidad algebraicamente y evita desperdiciar presupuesto de búsqueda.

7. Infraestructura de Computación en Servidor GPU

7.1. Arquitectura del clúster UGR

Los experimentos a escala completa se ejecutan en el clúster de computación GPU de la Universidad de Granada, con la siguiente configuración:

- **Nodo cabecera:** `ngpu.ugr.es` — punto de acceso SSH y gestor de colas SLURM.
- **Nodo de cómputo:** `titan` — equipado con GPUs NVIDIA (capacidad reportada: 11.90 GiB por GPU).
- **Almacenamiento:** `/mnt/homeGPU/egullon101/` — almacenamiento persistente de 239 TB compartidos.
- **Gestor de recursos:** SLURM (*Simple Linux Utility for Resource Management*).

7.2. Entorno Conda remoto

Se configuró un entorno Conda aislado en el almacenamiento GPU para evitar conflictos de dependencias:

```
/mnt/homeGPU/egullon101/conda_tfg/
```

Las dependencias principales instaladas incluyen: PyTorch (con CUDA), `momentfm`, `chronos-forecasting`, `timesfm`, `optuna`, `scikit-learn`, `pandas` y `matplotlib`.

7.3. Scripts SLURM de ejecución

Para posibilitar la ejecución desatendida y a escala de los experimentos, se estructuraron scripts bash controlados por SLURM. Estos scripts resuelven de forma nativa dos limitaciones críticas del entorno del clúster universitario:

1. **Límite de Cuota de Disco en el Directorio Home:** Las librerías de HuggingFace y los servidores de Jupyter descargan por defecto los pesos de los modelos fundacionales (que suman más de 5 GB en total) y los archivos de sesión temporal en la carpeta de usuario de la cabecera (`~/cache/`), la cual cuenta con una cuota restringida a 5 GB. Para evitar el fallo por desbordamiento (*Disk quota exceeded*), se redireccionan los directorios de configuración y caché al almacenamiento de red persistente y con alta cuota del clúster mediante variables de entorno.
2. **Liberación de Caché de GPU:** Dado que el recolector de PyTorch no devuelve de inmediato la memoria asignada al driver del sistema mientras el proceso de Python esté activo, se configuró la ejecución secuencial de notebooks en procesos independientes de Python, garantizando una GPU totalmente libre al inicio de cada modelo.

El script `run_foundation_server.sh` ilustra esta configuración:

```
1 #!/bin/bash
2 #SBATCH --job-name Foundation_TFG
3 #SBATCH --partition dios
4 #SBATCH --gres=gpu:1
5 #SBATCH --mem=32G
6 #SBATCH --time=12:00:00
7 #SBATCH --output=/mnt/homeGPU/egullonl01/tfg/Jupyter/slurm-%j.out
8 #SBATCH --error=/mnt/homeGPU/egullonl01/tfg/Jupyter/slurm-%j.out
9
10 export PATH="/opt/anaconda/anaconda3/bin:$PATH"
11 export PATH="/opt/anaconda/bin:$PATH"
12
13 # Redireccionar caches y configs para evitar problemas de cuota de disco
14 export IPYTHONDIR="/mnt/homeGPU/egullonl01/.ipython"
15 export JUPYTER_CONFIG_DIR="/mnt/homeGPU/egullonl01/.jupyter"
16 export JUPYTER_DATA_DIR="/mnt/homeGPU/egullonl01/.local/share/jupyter"
17 export JUPYTER_RUNTIME_DIR="/mnt/homeGPU/egullonl01/.local/share/jupyter/runtime"
18 export HF_HOME="/mnt/homeGPU/egullonl01/.cache/huggingface"
19 export XDG_CACHE_HOME="/mnt/homeGPU/egullonl01/.cache"
20
21 eval "$(conda shell.bash hook)"
22 conda activate /mnt/homeGPU/egullonl01/conda_tfg
23 cd /mnt/homeGPU/egullonl01/tfg/Jupyter
24
25 # Configurar SERVER_MODE de manera temporal en los notebooks
26 sed -i 's/SERVER_MODE = False/SERVER_MODE = True/g' 09_foundation_models_moment.ipynb
27 sed -i 's/SERVER_MODE = False/SERVER_MODE = True/g' 09_foundation_models_chronos.ipynb
28
29 # 1. Ejecutar MOMENT fine-tuning en un proceso aislado de Python
30 echo "Starting MOMENT..."
31 jupyter nbconvert --to notebook --execute \
32     --ExecutePreprocessor.kernel_name=python3 \
33     --inplace 09_foundation_models_moment.ipynb
34
35 # 2. Ejecutar Chronos zero-shot en su propio proceso limpio de Python
36 echo "Starting Chronos..."
37 jupyter nbconvert --to notebook --execute \
38     --ExecutePreprocessor.kernel_name=python3 \
39     --inplace 09_foundation_models_chronos.ipynb
40
41 # Restaurar estado original para Git
42 sed -i 's/SERVER_MODE = True/SERVER_MODE = False/g' 09
```

```

43  _foundation_models_moment.ipynb
    sed -i 's/SERVER_MODE = True/SERVER_MODE = False/g' 09
    _foundation_models_chronos.ipynb

```

Listing 5: Script SLURM completo para modelos fundacionales (run_foundation_server.sh).

Mecanismo SERVER_MODE. Todos los notebooks incluyen una variable `SERVER_MODE` que controla qué variante de modelo se carga. Los scripts SLURM la cambian automáticamente a `True` antes de la ejecución (para cargar los modelos grandes) y la revierten a `False` al finalizar (para mantener un estado limpio en Git). Los tres scripts comparten la configuración: partición `dios`, 1 GPU, 32 GB de RAM y 12 horas máximo. El script `run_timesfm_server.sh` sigue una estructura homóloga para procesar `TIMESFM` de forma aislada.

7.4. Pipeline de despliegue automático

El script `scratch/deploy_server.py` automatiza la sincronización de código entre el entorno local y el servidor remoto:

1. Conexión SSH al nodo cabecera (`ngpu.ugr.es`).
2. Subida de la librería completa `fatigueset-lib/` vía SFTP.
3. Subida de los notebooks y scripts SLURM.
4. Verificación de la integridad de los archivos transferidos.

7.5. Monitorización remota de jobs

Se desarrollaron múltiples scripts de monitorización bajo `scratch/`:

- `check_slurm_logs.py`: consulta `squeue` y muestra las últimas líneas de los logs de SLURM.
- `check_slurm_job_details.py`: ejecuta `scontrol show job` para obtener el estado detallado.
- `inspect_foundation_notebook.py`: descarga el notebook ejecutado del servidor y extrae las salidas de cada celda para diagnóstico remoto.
- `inspect_remote_output.py`: lista los archivos generados en el directorio `output/` del servidor.

8. Resultados Experimentales

8.1. Tabla comparativa global

La Tabla 5 presenta los resultados de todos los modelos evaluados. Las métricas son MAE (error absoluto medio), RMSE (raíz del error cuadrático medio) y R^2 (coeficiente de determinación). Para los modelos fundacionales con predicciones separadas por dimensión, se reportan las dos dimensiones explícitamente.

Nota metodológica sobre los modelos clásicos. Es sumamente importante destacar una discrepancia metodológica en los resultados de los modelos clásicos de Machine Learning. Mientras que las métricas de los modelos de Deep Learning (personalizados y fundacionales) corresponden a la evaluación sobre el conjunto de validación en validación cruzada (reflejando capacidad de generalización real), el MAE y RMSE de los modelos clásicos se evaluaron sobre el propio conjunto de entrenamiento. Esto introduce un sesgo optimista severo y explica sus errores aparentemente reducidos (p. ej., un MAE de 0.88 para Random Forest o 0.00 para el árbol de decisión). El R^2 negativo en la validación cruzada para estos mismos modelos clásicos confirma que sufren de un sobreajuste severo y carecen de capacidad real de generalización sobre sujetos no vistos.

Cuadro 5: Resultados comparativos globales de todos los modelos del TFG.

Modelo	Tipo	MAE Medio	RMSE Medio	R^2 Medio	Params
<i>Modelos clásicos de ML (validación cruzada KFold para R^2, entrenamiento para MAE/RMSE)[§]</i>					
Random Forest	Ensamble	0.88 [§]	1.04 [§]	-2.41	—
KNN (k=3)	Instancias	1.69 [§]	2.20 [§]	-3.57	—
Decision Tree	Árbol	0.00*	0.00*	1.00*	—
<i>Modelos de Deep Learning custom (MAE/RMSE/R^2 promediados sobre ambas dimensiones, validación CV)</i>					
Custom GRU (Opt.)	Recurrente	16.85	20.20	-0.62	72,866
Custom LSTM (Opt.)	Recurrente	17.02	20.44	-0.66	97,058
Custom PatchTST (Opt.)	Transformer	17.50	20.78	-0.68	162,882
Custom CNN-LSTM (Opt.)	Híbrido	17.54	20.61	-0.68	638,946
Custom TCN (Opt.)	Convolutacional	17.69	21.61	-0.86	175,298
Custom xLSTM (Opt.)	Recurrente	17.85	21.09	-0.74	323,522
Custom Transformer (Opt.)	Transformer	17.97	20.99	-0.76	139,310
Custom RNN (Baseline)	Recurrente	30.09	34.93	-5.07	1,890
<i>Modelos fundacionales (métricas de validación por dimensión, zero-shot/fine-tuning)</i>					
Chronos (chronos-t5-base)	Foundation	Fís: 14.11	Fís: 17.03	Fís: -0.45	710M [†]
		Men: 18.95	Men: 22.22	Men: -1.17	
TIMESFM 2.5	Foundation	Fís: 16.39	Fís: 20.15	Fís: -1.10	200M [†]
		Men: 22.38	Men: 26.71	Men: -1.43	
MOMENT (large)	Foundation	Fís: 24.64	Fís: 28.98	Fís: -3.88	2,050 [‡]
		Men: 36.68	Men: 41.37	Men: -8.07	

* Decision Tree con MAE=0 e $R^2 = 1,00$ indica sobreajuste severo (memorización completa del conjunto de entrenamiento).

[†] Parámetros totales del backbone preentrenado (0 parámetros entrenados, linear probe externo).

[‡] Parámetros entrenables de la cabeza de regresión (backbone de 341M congelado).

[§] Métricas de MAE y RMSE calculadas sobre el conjunto de entrenamiento (no de validación cruzada).

8.2. MOMENT: resultados por fold

La Tabla 6 muestra los resultados de MOMENT-1-large con fine-tuning de la cabeza de regresión, usando 5-fold GroupKFold por participante.

Cuadro 6: Resultados de MOMENT-1-large por fold (GroupKFold, 5 splits, 30 épocas).

Fold	MAE Fís.	RMSE Fís.	R^2 Fís.	MAE Men.	RMSE Men.	R^2 Men.
1	26.30	30.19	-3.08	29.38	31.41	-7.00
2	24.82	29.11	-2.66	44.46	48.61	-5.11
3	33.21	36.70	-4.52	52.77	56.53	-6.78
4	25.49	26.95	-8.55	27.42	28.08	-20.56
5	13.37	21.93	-0.59	29.37	42.23	-0.90
Media	24.64	28.98	-3.88	36.68	41.37	-8.07
$\pm$ Std	± 6.39	± 4.80	± 2.65	± 10.12	± 10.57	± 6.62

Configuración: batch size 32, learning rate 3×10^{-5} , early stopping con paciencia 10.
Tiempo total de fine-tuning: 5.209,7 s ($\approx$ 87 min).

Observaciones.

- El Fold 5 muestra el mejor rendimiento ($R_{\text{fís}}^2 = -0,59$, $R_{\text{men}}^2 = -0,90$), sugiriendo que los participantes de validación de ese fold tienen patrones más alineados con los del entrenamiento.
- El Fold 4 presenta el peor R^2 mental (-20,56), indicando una distribución de fatiga mental muy diferente en ese grupo de participantes.
- La alta varianza inter-fold (± 6.62 en R^2 mental) refleja la heterogeneidad individual inherente a FATIGUSET.

8.3. Chronos: métricas zero-shot y comparación de escalas

La Tabla 7 presenta los resultados probabilísticos y deterministas de CHRONOS utilizando las variantes `chronos-t5-tiny` (8M, ejecutado en local) y `chronos-t5-base` (710M, ejecutado en servidor).

Cuadro 7: Métricas probabilísticas y deterministas de las variantes de Chronos.

Métrica	Chronos-T5-tiny (8M)		Chronos-T5-base (710M)	
	Fatiga Física	Fatiga Mental	Fatiga Física	Fatiga Mental
MAE	17.25	23.40	14.11	18.95
RMSE	21.05	27.60	17.03	22.22
CRPS (media $\pm$ std)	12.59 ± 3.98	17.28 ± 6.30	10.17 ± 2.63	13.69 ± 4.88
Cobertura 90 %	68.4 %	63.2 %	78.9 %	73.8 %

8.4. TimesFM: métricas puntuales y probabilísticas

Cuadro 8: Resultados de TimesFM 2.5 (200M) con zero-shot + linear probe.

Métrica	Fatiga Física	Fatiga Mental
MAE	16.39	22.38
RMSE	20.15	26.71
R^2	-1.10	-1.43
CRPS (media $\pm$ std)	12.53 $\pm$ 3.93	17.30 $\pm$ 6.35
Cobertura 90 %	68.7 %	62.8 %
Parámetros entrenados	0 (probe externo)	
Tiempo total	536.7 s	

Observaciones.

- TIMESFM alcanza el mejor MAE global del proyecto (16.39 en fatiga física, 22.38 en mental), superando a los modelos de DL custom y a MOMENT.
- Las métricas probabilísticas de TIMESFM son muy similares a las de CHRONOS-tiny (CRPS física: 12.53 vs 12.59; cobertura 90 % física: 68.7 % vs 68.4 %), sugiriendo que ambos modelos producen distribuciones predictivas de calidad comparable.
- La cobertura del 90 % está por debajo del nominal (68–69 % en lugar del esperado 90 %), lo que indica que los intervalos de predicción son demasiado estrechos (subconfianza).

8.5. Optuna: resultados del experimento básico

El experimento básico de Optuna (sin optimizadores personalizados) se ejecutó con 3 trials y búsqueda sobre `hidden_size`, `dropout` y `lr`. Los mejores hiperparámetros encontrados fueron:

- `hidden_size`: 64
- `dropout`: 0.177
- `lr`: 0.00329
- Mejor MSE de validación: 1069.44

Los resultados detallados de la comparativa con los 7 modelos de DL se recogen en la Tabla 5. El experimento extendido con optimizadores personalizados (4 familias: Adam, AdamW, RMSprop, SGD) fue cancelado por exceder el límite de 12 horas del servidor (ver Sección 9.3).

8.6. Discusión y análisis comparativo

Los resultados experimentales revelan varias conclusiones clave:

1. **CHRONOS (chronos-t5-base)** es el mejor modelo del proyecto, logrando el menor error absoluto medio tanto para fatiga física (MAE de 14.11) como para fatiga mental (MAE de 18.95). Su paradigma zero-shot, que discretiza los valores numéricos mediante cuantiles y utiliza la potente base lingüística del T5, demuestra ser una excelente herramienta para extraer representaciones fisiológicas universales y robustas.
2. **TIMESFM 2.5 (200M)** muestra también un rendimiento muy competitivo (MAE físico: 16.39; mental: 22.38), superando a todos los modelos recurrentes. Esto confirma que los decoders de series temporales basados en parches o tokens discretos son superiores a los encoders o arquitecturas secuenciales clásicas.
3. **Los modelos de DL custom optimizados con Optuna** (LSTM, GRU, PatchTST) logran mejorar significativamente frente a sus baselines no optimizados. El MAE promedio de LSTM y GRU se redujo de 30 a 16.8 - 17.0 tras la optimización bayesiana de la tasa de aprendizaje y dropout, igualando la precisión del modelo TCN (MAE 17.6) y PatchTST (MAE 17.5).
4. **El fine-tuning de cabeza (MOMENT)** no mejora sobre el zero-shot en datasets pequeños. Con solo 660 ventanas y alta variabilidad individual, el fine-tuning de MOMENT (large) es vulnerable al sobreajuste (MAE físico: 24.64, mental: 36.68), rindiendo peor que los modelos zero-shot o redes custom optimizadas.
5. **Los valores de R^2 negativos** en casi todos los modelos indican que la predicción individual de fatiga sigue siendo un problema sumamente complejo debido a la gran variabilidad inter-sujeto de FATIGUSET. No obstante, la reducción sustancial del MAE por parte de Chronos (14.11 frente a la desviación estándar de la fatiga física de 16.52) representa un avance experimental significativo.
6. **La calibración probabilística de Chronos-base es notablemente superior:** logra coberturas del intervalo de predicción del 78.9 % (física) y 73.8 % (mental), un incremento considerable frente al 68.7 %/62.8 % de TimesFM o el 68.4 %/63.2 % de Chronos-tiny, evidenciando el efecto positivo de la escala del modelo en la estimación de la incertidumbre.

9. Problemas Técnicos y Soluciones

9.1. CUDA Out of Memory: diagnóstico y corrección

Problema. Al ejecutar el notebook `09_foundation_models.ipynb` en el servidor, MOMENT-1-large se entrenó correctamente en los 5 folds de validación cruzada, pero CHRONOS-large falló con el error:

```
CUDA out of memory. Tried to allocate 2.94 GiB. GPU 0 has a
total capacity of 11.90 GiB of which 210.81 MiB is free.
Of the allocated memory 11.50 GiB is allocated by PyTorch.
```

Causa raíz. La GPU asignada por SLURM tiene 11.90 GiB de capacidad (no 24 GiB como se esperaba inicialmente). Tras el fine-tuning de MOMENT, PyTorch retenía 11.50 GiB de tensores en la GPU. Aunque el backbone compartido se eliminó explícitamente al final de la función (`del backbone_shared`; `gc.collect()`; `torch.cuda.empty_cache()`), las activaciones intermedias del forward pass permanecían en memoria.

Análisis técnico. El problema subyacente es que en el bucle de entrenamiento:

```
1 model.train()
2 for xb, yb in train_loader:
3     pred = model(xb)          # Forward pass sin torch.no_grad()
4     loss = loss_fn(pred, yb)
5     loss.backward()
```

Aunque el backbone tiene `requires_grad=False`, al ejecutarse en modo `model.train()` sin `torch.no_grad()` alrededor de la llamada al backbone, PyTorch almacena las activaciones intermedias de las 24 capas Transformer para el grafo de autodiferenciación. Estas activaciones consumen varios GiB de memoria GPU.

Solución implementada. Se implementó una estrategia de **backbone compartido** para reducir el número de cargas del modelo de 5 (una por fold) a 1:

```
1 # Cargar backbone UNA sola vez
2 backbone_shared = MOMENTPipeline.from_pretrained(checkpoint,
3     ...)
4
5 for fold_idx, (train_idx, val_idx) in enumerate(kf.split(...)):
6     model = MOMENTFatigueRegressor(
7         ..., backbone=backbone_shared) # Reutilizar
8     model = model.to(device)
9     # ... entrenamiento ...
10    del model, optimizer
11    gc.collect(); torch.cuda.empty_cache()
12
13 # Liberar backbone al final
14 del backbone_shared
15 gc.collect(); torch.cuda.empty_cache()
```

Listing 6: Backbone compartido en `finetune_moment_kfold`.

Soluciones finales aplicadas. Se han implementado tres soluciones complementarias para resolver los problemas de memoria GPU:

1. **Inferencia sin gradientes:** Se envolvió el paso forward del backbone congelado en un bloque `with torch.no_grad()`: en `MOMENTFatigueRegressor.forward` (cuando `freeze_backbone` es `True`). Esto evitó de forma efectiva que PyTorch retuviera los grafos de activaciones en memoria.
2. **Separación de procesos secuenciales:** Dado que el distribuidor de caché de PyTorch (*caching allocator*) mantiene bloques asignados para acelerar futuras peticiones y no los devuelve al driver de la GPU mientras el proceso de Python esté vivo, se

dividió el notebook `09_foundation_models.ipynb` en dos notebooks independientes: `09_foundation_models_moment.ipynb` y `09_foundation_models_chronos.ipynb`. El script de ejecución de SLURM ejecuta ambos de forma secuencial en procesos de Python independientes. Al terminar el primer notebook, su proceso se destruye liberando por completo la memoria de la GPU, lo que permite al segundo notebook ejecutarse sobre una GPU totalmente vacía.

3. **Aislamiento de TimesFM:** Al igual que Chronos y MOMENT, el modelo de Google TIMESFM 2.5 (200M) se ejecuta en su propio notebook independiente `10_timesfm.ipynb` y mediante el script de lanzamiento `run_timesfm_server.sh`. Esto garantiza un entorno limpio y predecible de memoria, libre de cualquier tipo de fragmentación previa del caching allocator.

9.2. Exceso de Cuota de Disco en Directorio Home (Disk Quota Exceeded)

Problema. Durante la primera descarga de los backbones fundacionales preentrenados, la ejecución abortaba con un error de escritura en disco:

```
OSError: [Errno 122] Disk quota exceeded
```

Causa. Por defecto, la librería `transformers` de HuggingFace descarga y almacena los pesos de los modelos (MOMENT-large: ~ 1.5 GB, Chronos-base: ~ 2.8 GB, TimesFM: ~ 1 GB) en el directorio personal del usuario bajo el host cabecera (`~/cache/huggingface`). La cuota de almacenamiento para directorios de usuario del servidor universitario está limitada a 5 GB, lo que bloqueaba la descarga concurrente o consecutiva de los modelos fundacionales.

Solución implementada. Se redirigieron de forma explícita todas las variables de entorno de caché, ejecución y configuración al almacenamiento compartido persistente del clúster de computación, el cual posee una capacidad de almacenamiento de 239 TB y sin restricciones estrictas de cuota individual. En el inicio de los scripts SLURM, se establecen las siguientes redirecciones:

- `HF_HOME`: Redirigido a `/mnt/homeGPU/egullon101/.cache/huggingface` para almacenar de forma segura los pesos de HuggingFace.
- `XDG_CACHE_HOME`: Redirigido a `/mnt/homeGPU/egullon101/.cache` para el almacenamiento de caché general.
- `IPYTHONDIR`: Redirigido a `/mnt/homeGPU/egullon101/.ipython` para la caché de IPython.
- `JUPYTER_*`: Las variables de configuración y datos de Jupyter (`JUPYTER_CONFIG_DIR`, `JUPYTER_DATA_DIR`, `JUPYTER_RUNTIME_DIR`) se redirigieron a subcarpetas equivalentes en `/mnt/homeGPU/egullon101/`.

Esta reconfiguración evitó por completo los desbordamientos de cuota en el home de usuario, garantizando descargas de modelos sin fricciones.

9.3. Exceso de tiempo en SLURM (Optuna)

Problema. El Job 155622 (`experimento_optuna_optimizadores.ipynb`) fue cancelado por SLURM tras exceder las 12 horas configuradas:

```
slurmstepd: error: *** JOB 155622 ON titan CANCELLED AT
2026-07-03T19:52:18 DUE TO TIME LIMIT ***
```

Causa. El espacio de búsqueda del experimento con optimizadores personalizados es significativamente más grande que el experimento básico (8 familias $\times$ 4 optimizadores $\times$ múltiples hiperparámetros condicionales). Cada trial requiere entrenar un modelo completo con validación cruzada GroupKFold (3 splits), lo que multiplica el tiempo de ejecución por 3.

Solución propuesta. Reducir el número de trials y/o las épocas de entrenamiento para que el experimento complete dentro del límite de 12 horas de SLURM, y relanzar el job.

10. Conclusiones y Trabajo Futuro

10.1. Comparativa final de paradigmas

1. **Modelos fundacionales zero-shot** (TIMESFM, CHRONOS) alcanzan resultados competitivos sin necesidad de entrenamiento específico, validando su utilidad como extractores de características universales para dominios no vistos.
2. **El fine-tuning de cabeza** (MOMENT) no mejora necesariamente sobre el zero-shot en datasets pequeños. Con solo 660 ventanas y alta variabilidad individual, el fine-tuning es vulnerable al sobreajuste a pesar del pequeño número de parámetros entrenables (2.050).
3. **Los modelos custom bien diseñados** (TCN, PatchTST) siguen siendo competitivos, especialmente cuando la arquitectura se adapta específicamente al problema (convoluciones dilatadas para señales fisiológicas, parches para capturas temporales multi-escala).
4. **La heterogeneidad individual** de FATIGUESet representa el mayor desafío experimental, como evidencia la alta varianza inter-fold y los valores negativos de R^2 en todos los modelos.

10.2. Tareas pendientes

1. **Optuna con optimizadores:** reducir el presupuesto de búsqueda (trials y/o épocas) y relanzar el experimento extendido dentro del límite de 12 horas.
2. **Calibración probabilística avanzada:** explorar técnicas de calibración post-hoc (Platt scaling, regresión isotónica) para elevar la cobertura empírica al 90 % nominal.
3. **Análisis por participante:** evaluar métricas condicionadas al individuo para cuantificar la variabilidad inter-sujeto y proponer modelos personalizados de regresión.

Referencias

- [1] Kalanadhabhatta, M., Zou, A., Rajagopalan, A., et al. (2022). *FatigueSet: A Multi-modal Dataset for Modeling Mental and Physical Fatigue*. arXiv:2203.06924.
- [2] Kottapalli, S. R. K., Hubli, K., Chandrashekhara, S., et al. (2025). *Foundation Models for Time Series: A Survey*. arXiv.
- [3] Beck, M., Pöppel, K., Spanring, M., et al. (2024). *xLSTM: Extended Long Short-Term Memory*. arXiv:2405.20455.
- [4] Goswami, M., Szafer, K., Choudhry, A., et al. (2024). *MOMENT: A Family of Open Time-series Foundation Models*. International Conference on Machine Learning (ICML).
- [5] Ansari, A. F., Stella, L., Turkmen, C., et al. (2024). *Chronos: Learning the Language of Time Series*. arXiv:2403.07815.
- [6] Das, A., Kong, W., Leach, A., et al. (2024). *A Decoder-only Foundation Model for Time-series Forecasting*. International Conference on Machine Learning (ICML).
- [7] Gneiting, T. & Raftery, A. E. (2007). *Strictly Proper Scoring Rules, Prediction, and Estimation*. Journal of the American Statistical Association, 102(477), 359–378.